

Exercices supplémentaires semaine 13

Sophie Baillargeon, Université Laval

6 avril 2021

Question 1

Créez une fonction R qui prend en entrée un vecteur numérique $\mathbf{x}$ ainsi qu'un deuxième argument nommé $\mathbf{dg}$. Cet argument représente le degré d'un polynôme. Il s'agit d'un entier non-nul. Le but de la fonction est de créer une matrice à $\text{length}(\mathbf{x})$ lignes et $\mathbf{dg}$ colonnes, dont la première colonne est une copie de $\mathbf{x}$, la deuxième colonne contient les éléments de $\mathbf{x}$ à la puissance 2, la troisième colonne contient les éléments de $\mathbf{x}$ à la puissance 3, et ainsi de suite jusqu'à la puissance $\mathbf{dg}$. Par exemple, pour $\mathbf{x} = 1:5$ et $\mathbf{dg} = 4$, la fonction devrait retourner :

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    1    1    1
## [2,]    2    4    8   16
## [3,]    3    9   27   81
## [4,]    4   16   64  256
## [5,]    5   25  125  625
```

Si nous ajoutons à cette matrice une première colonne contenant uniquement des 1 (ce que nous ne voulons pas faire ici), nous obtiendrions la matrice de design d'un modèle de régression polynômiale de degré $\mathbf{dg}$, incluant une ordonnée à l'origine, avec les observations de la variable explicative $\mathbf{x}$.

Créez différentes versions de cette fonction et identifiez la version la plus rapide en terme de temps d'exécution. Vos différentes versions doivent permettre de comparer :

- l'utilisation d'un objet de taille croissante à l'utilisation d'un objet de taille fixe dans une boucle,
- l'utilisation de l'opérateur $\wedge$ (exposant) à l'utilisation de l'opérateur $*$ (multiplication),
- une boucle à du calcul vectoriel ou matriciel,
- une boucle à l'utilisation d'une fonction de la famille des `apply`,
- une boucle à l'utilisation de métaprogrammation,
- votre fonction la plus performante à la fonction `poly`.

Cet exercice est inspiré de l'exercice 14.2.3 de

Matloff, N. (2011). *The Art of R Programming : A Tour of Statistical Software Design*, No Starch Press.

Question 2

Reprenons l'exemple suivant des notes de cours : simulons une expérience aléatoire pour compter le nombre de fois que nous devons tirer un dé afin d'atteindre une somme des valeurs obtenues supérieure ou égale à `somme_visee`.

Le code le plus rapide que nous avons développé est le suivant :

```
somme_de_vecteur_fixe <- function(somme_visee = 50000){
  somme <- 0
  resultats <- integer(somme_visee)
  i <- 0
  while (somme < somme_visee) {
    tirage <- sample(1:6, size = 1)
    somme <- somme + tirage
    i <- i + 1
    resultats[i] <- tirage
  }
  resultats[1:i]
}
```

Y aurait-il moyen d'avoir un code encore plus rapide? Essayez différentes stratégies et comparez les temps d'exécution. Par exemple, nous pourrions :

- simuler tous les tirages du dé avec un seul appel à la fonction `sample` (donc sortir de la boucle l'appel à la fonction `sample`),
- éviter complètement d'utiliser une boucle grâce à la fonction `cumsum`.

Question 3

Créez une fonction qui calcule une mesure de tendance centrale sur les observations numériques d'une variable. Cette fonction doit prendre en entrée :

- les observations,
- l'argument `robust` qui peut prendre les valeurs :
 - `TRUE` : la mesure calculée est une médiane,
 - `FALSE` (défaut) : la mesure calculée est une moyenne.

Écrivez différentes versions de cette fonction, respectant les contraintes suivantes :

1. n'utilise pas de métaprogrammation,
2. utilise la fonction `get`,
3. utilise la fonction `do.call`,
4. utilise les fonctions `call` et `eval`.

Question 4

Expliquez pourquoi l'assignation suivante ne fonctionne pas,

```
assign(x = moyenne, value = mean(quakes$mag))
```

```
## Error in assign(x = moyenne, value = mean(quakes$mag)): object 'moyenne' not found
```

et réécrivez là de deux façons différentes (qui fonctionnent toutes les deux).